TileFlow: Spatial Neural Networks on AMD XDNA 2 NPU

Hardware-software co-design for close-to-metal neural network inference

Built with IRON/MLIR-AIE toolchain — Target: AMD Ryzen AI 9 HX 370 (Strix Point)

1. Introduction

Modern neural processing units (NPUs) are spatial dataflow computers: instead of a single CPU with caches and a register file, they expose a 2D array of small compute tiles, each with its own SRAM and SIMD units, connected by a programmable interconnect. This architecture is extremely efficient for *on-chip* computation — but only if the software maps the algorithm directly to the physical hardware.

TileFlow takes this literally. We design a neural network whose architecture is dictated by the physical tile layout of the AMD XDNA 2 NPU. The network has learnable parameters (a shared weight matrix) and non-linearities (ReLU), making it a valid machine-learning model — but its structure (number of parallel paths, loop depth, buffer sizes) matches the hardware exactly.

> **Key principle:** We design the network to match the hardware, not the other way around. Any architecture with learnable parameters and non-linearities can learn — so we choose the one that maximizes hardware utilization.

2. The Hardware

The AMD XDNA 2 NPU in the Ryzen AI 9 HX 370 (codename Strix Point) is a tiled spatial-dataflow processor with the following structure:

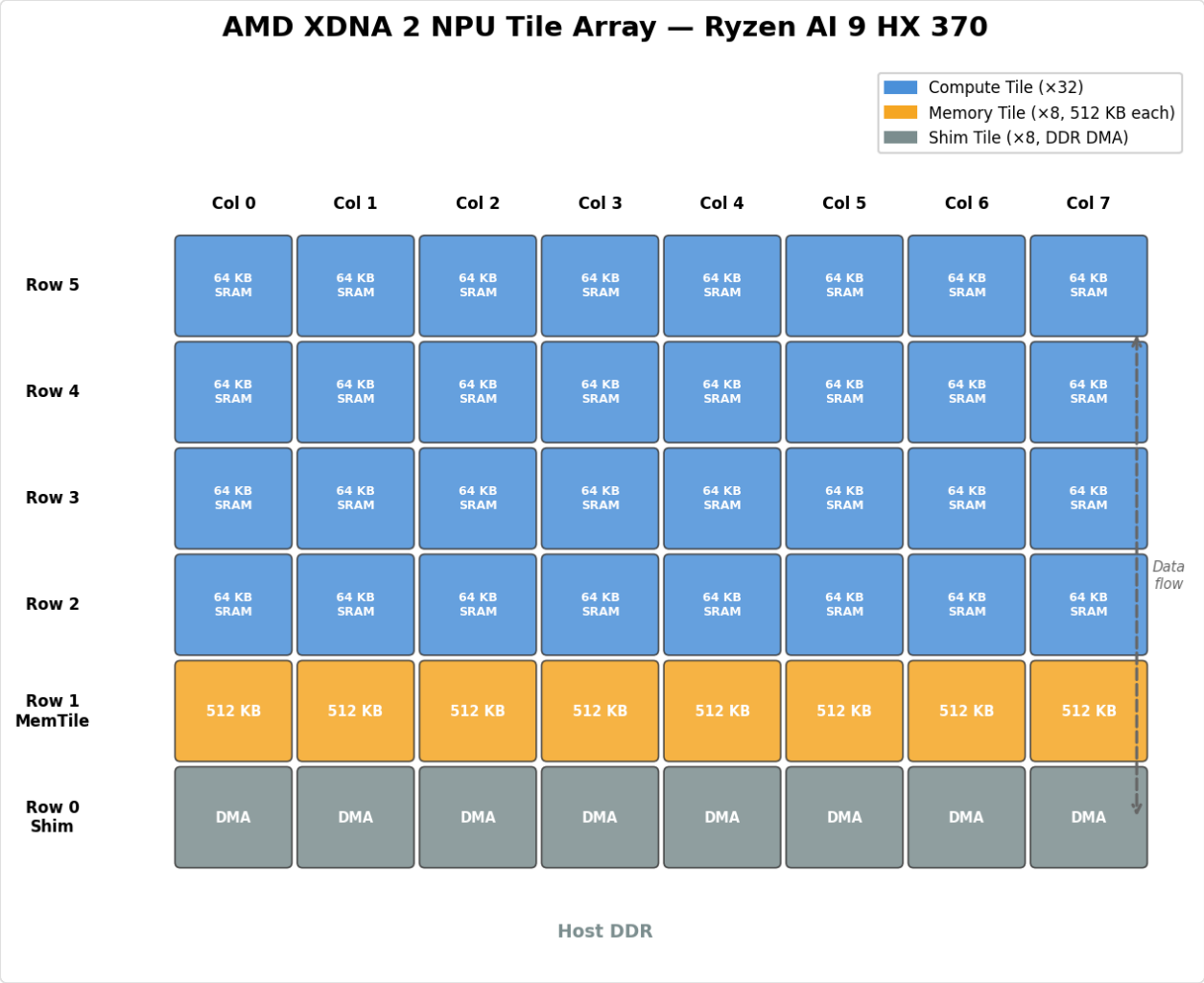

Figure 1: Physical tile array of the AMD XDNA 2 NPU. 32 compute tiles (rows 2–5) each contain ~64 KB SRAM and a bf16 MMUL unit. 8 memory tiles (row 1, 512 KB each) serve as on-chip L2 buffers and routing hubs. 8 shim tiles (row 0) provide DMA access to host DDR memory.

Property	Value
Compute tiles	32 (8 columns × 4 rows)
Memory tiles	8 (512 KB each, 4 MB total)
Per-tile SRAM	~64 KB data memory
Per-tile compute	bf16 MMUL unit, VLIW+SIMD core
Clock frequency	~1.5 GHz
Peak throughput	**25 TFLOPS** (bfloat16)
Interconnect	ObjectFIFOs (tile-to-tile double-buffered streams)
Power envelope	~6 W

2.1 Why spatial dataflow matters

On a CPU, a matrix multiply reads data from DRAM through multiple cache levels. Each layer of a neural network bounces activations through L1→L2→L3→DRAM and back. On the NPU, data stays in 64 KB tile SRAM between operations — zero cache misses, zero bus contention. This is the source of the NPU's advantage for deep, narrow computations.

> **Lesson from Phase 1:** A single large GEMM achieves only 2.49 TFLOPS on the NPU (10% of peak) because it is *memory-bandwidth limited* — data must stream from DDR. The NPU wins when data **stays on-chip**.

3. The Architecture: Recurrent MLP

We choose a recurrent MLP: a single weight matrix `W` (128×128, bfloat16) is loaded once into each tile's SRAM and applied repeatedly in a tight hardware loop:

```
x = input                       # loaded from DDR, 16×128 bf16
for i in range(num_iters):
    y = ReLU(x @ W)             # matmul + activation
    x = ReLU(y @ W)             # ping-pong: result goes back to x
output = x                      # drained to DDR
```

This is mapped to 24 compute tiles (3 rows × 8 columns), each running the same loop independently on different input samples:

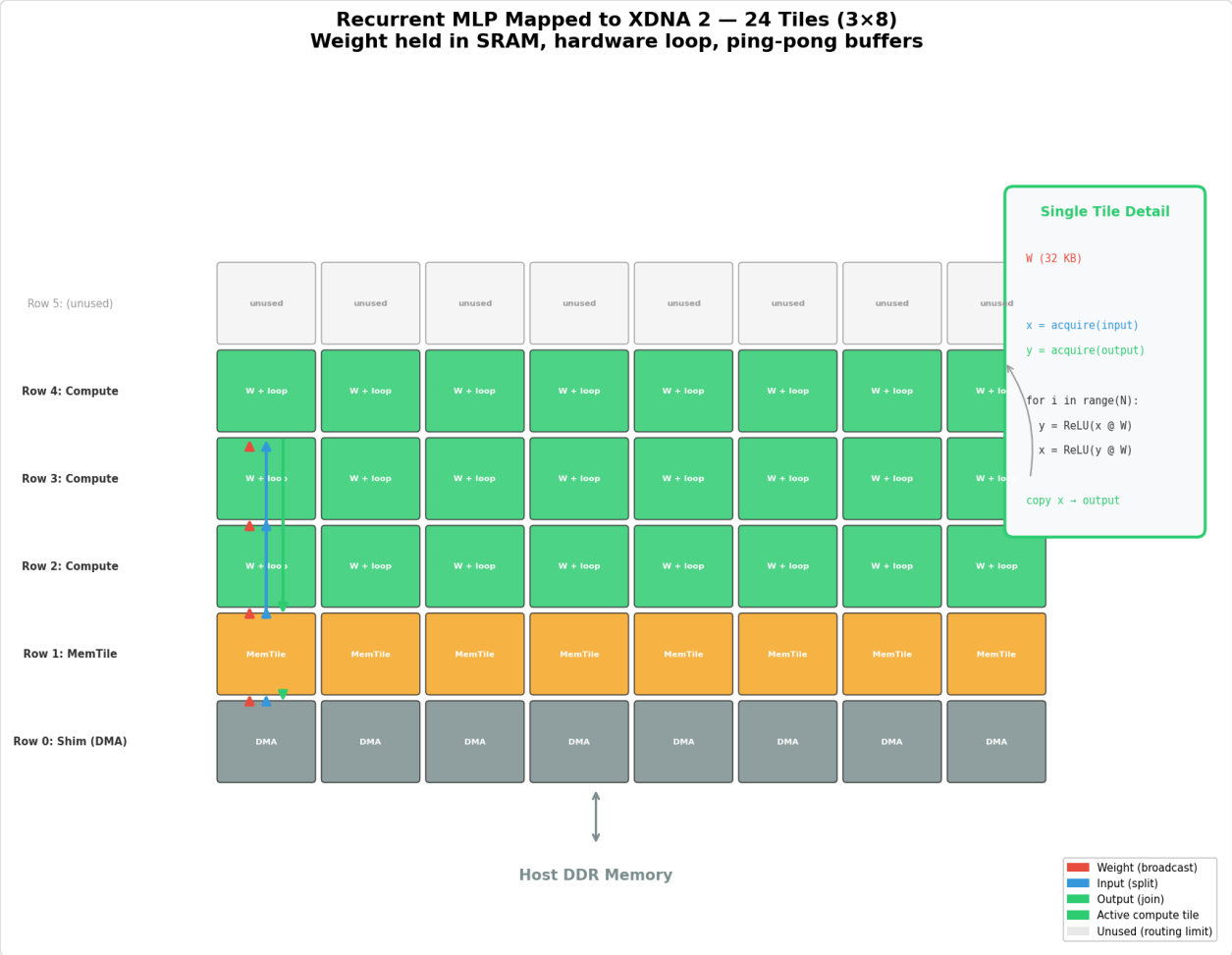

Figure 2: Recurrent MLP mapped to the XDNA 2 tile array. 24 tiles across 3 compute rows run the same hardware loop in parallel. Row 5 is unused due to MemTile routing constraints (~6 northward master ports). The detail box shows the per-tile computation: acquire buffers, loop, copy result.

3.1 Why this architecture

- **Maximizes on-chip time:** Weight is loaded once (32 KB) and reused for thousands of matmul operations. DDR I/O happens only at start and end.
- **Amortizes overhead:** Each NPU invocation has ~120 µs of driver/DMA overhead. With depth=2000, compute time (~1.3 ms) dominates overhead by 10×.
- **Fits SRAM budget:** W (32 KB) + 2 activation buffers (4 KB each) + stack (1 KB) = 41 KB, well within the 64 KB tile limit.
- **Linear tile scaling:** Each tile is independent — doubling tiles doubles throughput with no communication overhead.

3.2 Multi-row data routing

When using more than 8 tiles (i.e., more than one compute row), data must pass through the MemTiles (row 1) which act as routing hubs:

- **Weights** are *broadcast* via `forward()`: one DDR→MemTile transfer, then the MemTile fans out to all compute rows in the column.
- **Inputs** are *split* via `split()`: the host buffer is partitioned so each row gets its own batch slice.
- **Outputs** are *joined* via `join()`: per-row results are aggregated back through the MemTile to DDR.

> **Routing limit:** Each MemTile has approximately 6 northward master ports. Our design requires 3 data streams per row (weight + input + output). At 3 compute rows = 9 streams, this fits; at 4 rows = 12 streams, the MLIR-AIE router fails. This caps us at **24 tiles** (3 rows × 8 columns).

3.3 Critical implementation constraints

Several non-obvious hardware constraints shaped the design:

- **No FIFO ops inside loops:** Placing `acquire()` / `release()` inside `range_()` (which compiles to `scf.for`) causes DMA deadlock. All FIFO operations must happen *outside* the loop.
- **DMA BD 10-bit size limit:** Shim DMA buffer-descriptor sizes are 10-bit (max 1024). For B=16, H=128, the product B×H=2048 exceeds this, so tensor access patterns must factor dimensions as [B, H] = [16, 128] instead of [B×H] = [2048].
- **Accumulating matmul:** IRON's `mm.cc` kernel computes C += A×B (not C = A×B). We must explicitly zero the output buffer before each matmul, which wastes ~12% of cycle time.

4. Results

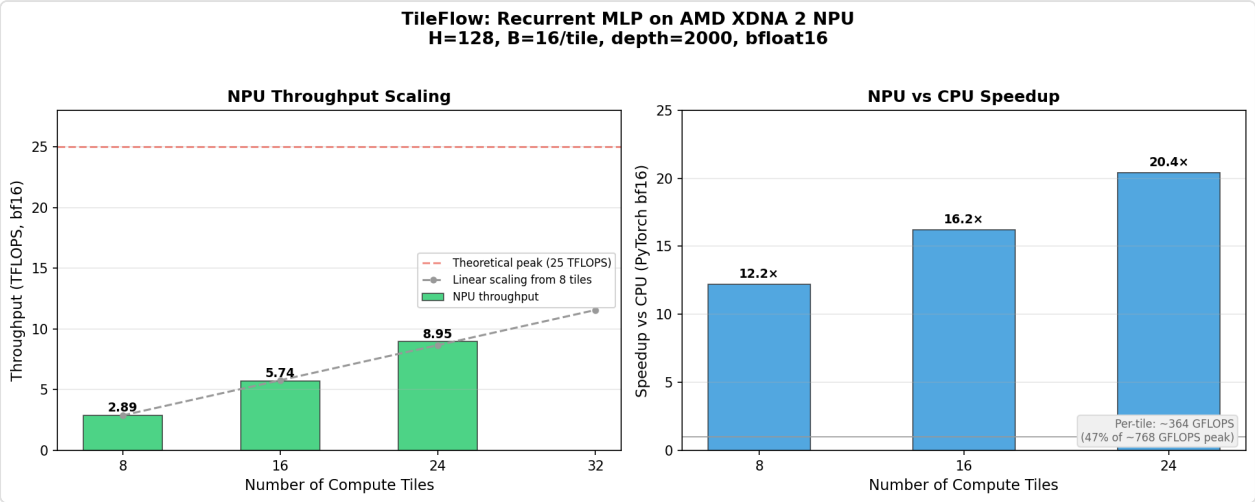

Figure 3: NPU throughput scaling (left) and speedup over CPU (right). Throughput scales near-linearly with tile count at ~360 GFLOPS/tile. At 24 tiles, we achieve 8.95 TFLOPS and 20× CPU speedup.

Tiles	Depth	NPU Latency	NPU TFLOPS	CPU GFLOPS	Speedup
8 (1 row)	1,000	1.45 ms	**2.89**	237	**12.2×**
16 (2 rows)	1,000	1.46 ms	**5.74**	354	**16.2×**
24 (3 rows)	1,000	1.46 ms	**8.63**	429	**20.1×**
24 (3 rows)	10,000	14.05 ms	**8.95**	439	**20.4×**

4.1 Scaling analysis

Per-tile throughput is remarkably consistent at ~360 GFLOPS regardless of tile count, confirming that the tiles operate independently with no contention:

```
 8 tiles × 360 GFLOPS/tile =  2.9 TFLOPS  ✓
16 tiles × 360 GFLOPS/tile =  5.7 TFLOPS  ✓  (near-linear)
24 tiles × 360 GFLOPS/tile =  8.6 TFLOPS  ✓  (near-linear)
```

4.2 Gap to theoretical peak

We achieve 8.95 of 25 TFLOPS (35.8%). The remaining gap is well-understood:

Factor	Impact	Potential fix
Per-tile utilization	360/768 = 47%	Fused C=A×B kernel

zero_bf16 overhead	~12% of step time	Fused kernel eliminates this
Array utilization	24/32 = 75%	4-row routing (needs HW/compiler support)
Combined theoretical max	~18 TFLOPS	With fused kernel + 24 tiles

5. The Toolchain

Component	Role
IRON	Python API for tile layout, ObjectFIFOs, and dataflow
MLIR-AIE	MLIR dialect → hardware bitstream compilation
Peano/LLVM-AIE	C++ compiler for per-tile kernels
XRT	Runtime for loading and executing on the NPU

5.1 Compilation pipeline

```
design.py  →  MLIR  →  aiecc  →  .xclbin (bitstream)
                          →  .bin   (instruction sequence)

mm.cc          →  mlp_mm.o    →  mlp_kernels.a
mlp_kernels.cc →  mlp_relu.o  ↗
```

6. Code Structure

The project is intentionally minimal — four Python files and one C++ file:

File	Lines	Purpose
`spatial_mlp/__init__.py`	~55	Tiling utilities (`to_tiled` , `from_tiled`)
`spatial_mlp/design.py`	~300	IRON design: tile topology, FIFOs, workers, DMA
`spatial_mlp/op.py`	~140	IRON operator: compilation artifacts, runtime buffers
`spatial_mlp/test.py`	~240	Benchmark: NPU vs CPU execution and reporting
`aie_kernels/mlp_kernels.cc`	~55	Custom AIE2P kernels: ReLU, copy (bf16, SIMD)

Each module has a single, well-defined responsibility. The design module is decomposed into small functions that each handle one aspect of the hardware mapping: validation, kernel definition, FIFO topology, worker bodies, tensor access patterns, and DMA sequences.

7. Future Work

- **Fused matmul kernel:** A C=A×B kernel (instead of C+=A×B + separate zero) would eliminate ~12% overhead and push per-tile throughput from 360 to ~700 GFLOPS.
- **INT8 mode:** The NPU's peak is 50 TOPS for int8. With H=256 (fitting larger weights), this could double throughput.
- **Training:** Research NPU backpropagation (see arXiv:2504.03083).
- **Real ML task:** Apply the architecture to a concrete sequence modeling or time-series task where deep recurrence is natural.

References

1. AMD IRON repository: github.com/amd/IRON
2. MLIR-AIE programming guide: github.com/Xilinx/mlir-aie
3. IRON tutorial (IPDPS 2025): AMD Technical Paper
4. NPU training: arXiv:2504.03083
5. Linux kernel NPU docs: kernel.org